TASK I

TO-DO LIST

A To-Do List application is a useful project that helps users manage and organize their tasks efficiently. This project aims to create a command-line or GUI-based application using Python, allowing users to create, update, and track their to-do lists

```
import os


FILENAME = "tasks.txt"


def load_tasks():
    tasks = []
    if os.path.exists(FILENAME):
        with open(FILENAME, "r") as file:
            for line in file:
                task, status = line.strip().split("|")
                tasks.append({"task": task, "done": status == "True"})
    return tasks


# Save tasks to filedef save_tasks(tasks):
    with open(FILENAME, "w") as file:
        for task in tasks:
            file.write(f"{task['task']}|{task['done']}\n")
def view_tasks(tasks):
    if not tasks:
        print("\nNo tasks found.\n")
        return
```

```
    print("\nYour To-Do List:")
    for i, task in enumerate(tasks, start=1):
        status = "✔" if task["done"] else "✘"
        print(f"{i}. [{status}] {task['task']}")
    print()

def add_task(tasks):
    task_name = input("Enter new task: ")
    tasks.append({"task": task_name, "done": False})
    print("Task added successfully!\n")

def complete_task(tasks):
    view_tasks(tasks)
    try:
        task_num = int(input("Enter task number to mark as completed: "))
        tasks[task_num - 1]["done"] = True
        print("Task marked as completed!\n")
    except (ValueError, IndexError):
        print("Invalid task number!\n")

def delete_task(tasks):
    view_tasks(tasks)
    try:
        task_num = int(input("Enter task number to delete: "))
        removed = tasks.pop(task_num - 1)
```

```
        print(f"Deleted task: {removed['task']}\n")
    except (ValueError, IndexError):
        print("Invalid task number!\n")

def main():
    tasks = load_tasks()

    while True:
        print("===== TO-DO LIST MENU =====")
        print("1. View Tasks")
        print("2. Add Task")
        print("3. Complete Task")
        print("4. Delete Task")
        print("5. Exit")

        choice = input("Choose an option (1-5): ")

        if choice == "1":
            view_tasks(tasks)
        elif choice == "2":
            add_task(tasks)
        elif choice == "3":
            complete_task(tasks)
        elif choice == "4":
            delete_task(tasks)
```

```
        elif choice == "5":
            save_tasks(tasks)
            print("Tasks saved. Goodbye!")
            break
        else:
            print("Invalid choice. Try again.\n")

if __name__ == "__main__":
    main()
```

TASK 2

CALCULATOR

Design a simple calculator with basic arithmetic operations. Prompt the user to input two numbers and an operation choice.

Perform the calculation and display the result.

```
num1 = float(input("Enter first number: "))

num2 = float(input("Enter second number: "))


print("\nChoose operation:")

print("1. Addition (+)")

print("2. Subtraction (-)")

print("3. Multiplication (*)")

print("4. Division (/)")


choice = input("Enter choice (1/2/3/4): ")


if choice == '1':

    result = num1 + num2

    print("Result:", result)


elif choice == '2':

    result = num1 - num2

    print("Result:", result)


elif choice == '3':
```

```
    result = num1 * num2
    print("Result:", result)

elif choice == '4':
    if num2 != 0:
        result = num1 / num2
        print("Result:", result)
    else:
        print("Error! Division by zero is not allowed.")

else:
    print("Invalid choice!")
```

TASK 3

PASSWORD GENERATOR

A password generator is a useful tool that generates strong and random passwords for users. This project aims to create a password generator application using Python, allowing users to specify the length and complexity of the password.

User Input: Prompt the user to specify the desired length of the password.

Generate Password: Use a combination of random characters to generate a password of the specified length.

Display the Password: Print the generated password on the screen

```
import random

import string


length = int(input("Enter desired password length: "))


letters = string.ascii_letters      # a-z and A-Z

digits = string.digits              # 0-9

symbols = string.punctuation        # special characters


all_characters = letters + digits + symbols


password = ""

for i in range(length):

    password += random.choice(all_characters)


print("Generated Password:", password)
```

TASK 4

Rock-Paper-Scissors Game

User Input: Prompt the user to choose rock, paper, or scissors.

Computer Selection: Generate a random choice (rock, paper, or scissors) for the computer.

Game Logic: Determine the winner based on the user's choice and the computer's choice.

Rock beats scissors, scissors beat paper, and paper beats rock.

Display Result: Show the user's choice and the computer's choice.

Display the result, whether the user wins, loses, or it's a tie.

Score Tracking (Optional): Keep track of the user's and computer's scores for multiple rounds.

Play Again: Ask the user if they want to play another round.

User Interface: Design a user-friendly interface with clear instructions and feedback.

```
import random


print("===== ROCK - PAPER - SCISSORS GAME =====")
print("Instructions:")
print("Type 'rock', 'paper', or 'scissors' to play.")
print("Type 'exit' anytime to quit.\n")


user_score = 0
computer_score = 0


choices = ["rock", "paper", "scissors"]


while True:
    user_choice = input("Enter your choice: ").lower()
```

```
if user_choice == "exit":
    print("\nThanks for playing!")
    print(f"Final Score -> You: {user_score} | Computer: {computer_score}")
    break

if user_choice not in choices:
    print("Invalid choice! Please try again.\n")
    continue

computer_choice = random.choice(choices)

print(f"\nYou chose: {user_choice}")
print(f"Computer chose: {computer_choice}")

if user_choice == computer_choice:
    print("It's a tie!")

elif (user_choice == "rock" and computer_choice == "scissors") or \
     (user_choice == "scissors" and computer_choice == "paper") or \
     (user_choice == "paper" and computer_choice == "rock"):
    print("You win this round!")
    user_score += 1

else:
```

```
    print("Computer wins this round!")
    computer_score += 1

print(f"Score -> You: {user_score} | Computer: {computer_score}\n")

play_again = input("Do you want to play again? (yes/no): ").lower()
if play_again != "yes":
    print("\nGame Over!")
    print(f"Final Score -> You: {user_score} | Computer: {computer_score}")
    break

print("\n---------------------------------\n")
```

TASK 5

Contact Book

Contact Information: Store name, phone number, email, and address for each contact.

Add Contact: Allow users to add new contacts with their details.

View Contact List: Display a list of all saved contacts with names and phone numbers. Contact: Implement a search function to find contacts by name or phone number.

Search

Update Contact: Enable users to update contact details.

Delete Contact: Provide an option to delete a contact.

User Interface: Design a user-friendly interface for easy interaction.

```
contacts = []


sdef add_contact():
    print("\n--- Add New Contact ---")
    name = input("Enter Name: ")
    phone = input("Enter Phone Number: ")
    email = input("Enter Email: ")
    address = input("Enter Address: ")


    contact = {
        "name": name,
        "phone": phone,
        "email": email,
        "address": address
    }
```

```
    contacts.append(contact)
    print("Contact added successfully!\n")



def view_contacts():
    print("\n--- Contact List ---")
    if not contacts:
        print("No contacts found.\n")
        return

    for i, contact in enumerate(contacts, start=1):
        print(f"{i}. {contact['name']} - {contact['phone']}")
    print()

def search_contact():
    print("\n--- Search Contact ---")
    search = input("Enter name or phone number to search: ").lower()
    found = False

    for contact in contacts:
        if search in contact["name"].lower() or search in contact["phone"]:
            print("\nContact Found:")
            print(f"Name: {contact['name']}")
            print(f"Phone: {contact['phone']}")
```

```
            print(f"Email: {contact['email']}")
            print(f"Address: {contact['address']}")
            found = True

    if not found:
        print("No matching contact found.")
    print()



def update_contact():
    view_contacts()
    try:
        choice = int(input("Enter contact number to update: "))
        contact = contacts[choice - 1]

        print("\nLeave blank to keep current value.")

        new_name = input(f"New Name ({contact['name']}): ")
        new_phone = input(f"New Phone ({contact['phone']}): ")
        new_email = input(f"New Email ({contact['email']}): ")
        new_address = input(f"New Address ({contact['address']}): ")

        if new_name:
            contact["name"] = new_name
        if new_phone:
```

```
            contact["phone"] = new_phone
        if new_email:
            contact["email"] = new_email
        if new_address:
            contact["address"] = new_address

        print("Contact updated successfully!\n")

    except (ValueError, IndexError):
        print("Invalid selection!\n")


def delete_contact():
    view_contacts()
    try:
        choice = int(input("Enter contact number to delete: "))
        removed = contacts.pop(choice - 1)
        print(f"Deleted contact: {removed['name']}\n")
    except (ValueError, IndexError):
        print("Invalid selection!\n")


# Main Menu
def main():
    while True:
```

```
print("===== CONTACT BOOK MENU =====")
print("1. Add Contact")
print("2. View Contacts")
print("3. Search Contact")
print("4. Update Contact")
print("5. Delete Contact")
print("6. Exit")

choice = input("Choose an option (1-6): ")

if choice == "1":
    add_contact()
elif choice == "2":
    view_contacts()
elif choice == "3":
    search_contact()
elif choice == "4":
    update_contact()
elif choice == "5":
    delete_contact()
elif choice == "6":
    print("Exiting Contact Book. Goodbye!")
    break
else:
    print("Invalid choice! Try again.\n")
```

```
if __name__ == "__main__":
    main()
```

.